

```
from google.colab import files
uploaded=files.upload()
print(uploaded)
```

 Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving SupplyChainEmissionFactorsforUSIndustriesCommodities.xlsx to SupplyChainEmissionFactorsforUSIndustriesCommodities (1).xlsx

{'SupplvChainEmissionFactorsforUSIndustriesCommodities (1).xlsx': b'PK\x03\x04\x14\x00\x00\x00\x08\x00\x00\x00!\x00\x85F\xa6~\x05\x02\x0e'}

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import joblib
```

Step 2: Load Dataset

```
excel_file = 'SupplyChainEmissionFactorsforUSIndustriesCommodities.xlsx'
years = range(2010, 2017)
```

```
years[2]
```

 2012

```
df_1 = pd.read_excel(excel_file, sheet_name=f'{years[0]}_Detail_Commodity')
df_1.head()
```



	Commodity Code	Commodity Name	Substance	Unit	Supply Chain Emission Factors without Margins	Margins of Supply Chain Emission Factors	Supply Chain Emission Factors with Margins	Unnamed: 7	ReliabilityScore of Factors without Margins	DQ TemporalCorrelation of Factors without Margins	DQ Geographical of Factors without Margins
0	1111A0	Fresh soybeans, canola, flaxseeds, and other o...	carbon dioxide	kg/2018 USD, purchaser price	0.398	0.073	0.470	NaN	4	3	
1	1111A0	Fresh soybeans, canola, flaxseeds, and other o...	methane	kg/2018 USD, purchaser price	0.001	0.001	0.002	NaN	4	3	
2	1111A0	Fresh soybeans, canola, flaxseeds, and other o...	nitrous oxide	kg/2018 USD, purchaser price	0.002	0.000	0.002	NaN	4	3	
3	1111A0	Fresh soybeans, canola, flaxseeds, and other o...	other GHGs	kg CO2e/2018 USD, purchaser price	0.002	0.000	0.002	NaN	3	3	
4	1111B0	Fresh wheat, corn, rice, and other grains	carbon dioxide	kg/2018 USD, purchaser price	0.659	0.081	0.740	NaN	4	3	

```
df_2 = pd.read_excel(excel_file, sheet_name=f'{years[0]}_Detail_Industry')
df_2.head()
```



	Industry Code	Industry Name	Substance	Unit	Supply Chain Emission Factors without Margins	Margins of Supply Chain Emission Factors	Supply Chain Emission Factors with Margins	Unnamed: 7	ReliabilityScore of Factors without Margins	DQ TemporalCorrelation of Factors without Margins	DQ Geographical of Factors without Margins
0	1111A0	Oilseed farming	carbon dioxide	kg/2018 USD, purchaser price	0.414	0.073	0.487	NaN	4	3	
1	1111A0	Oilseed farming	methane	kg/2018 USD, purchaser price	0.001	0.001	0.002	NaN	4	3	
2	1111A0	Oilseed farming	nitrous oxide	kg/2018 USD, purchaser price	0.002	0.000	0.002	NaN	4	3	
3	1111A0	Oilseed farming	other GHGs	kg CO2e/2018 USD, purchaser price	0.002	0.000	0.002	NaN	3	3	
4	1111B0	Grain farming	carbon dioxide	kg/2018 USD, purchaser price	0.680	0.082	0.762	NaN	4	3	

```
all_data = []
```

```
for year in years:
```

```
try:
    df_com = pd.read_excel(excel_file, sheet_name=f'{year}_Detail_Commodity')
    df_ind = pd.read_excel(excel_file, sheet_name=f'{year}_Detail_Industry')

    df_com['Source'] = 'Commodity'
    df_ind['Source'] = 'Industry'
    df_com['Year'] = df_ind['Year'] = year

    df_com.columns = df_com.columns.str.strip()
    df_ind.columns = df_ind.columns.str.strip()

    df_com.rename(columns={
        'Commodity Code': 'Code',
        'Commodity Name': 'Name'
    }, inplace=True)

    df_ind.rename(columns={
        'Industry Code': 'Code',
        'Industry Name': 'Name'
    }, inplace=True)

    all_data.append(pd.concat([df_com, df_ind], ignore_index=True))

except Exception as e:
    print(f"Error processing year {year}: {e}")

all_data[3]
```



	Code	Name	Substance	Unit	Supply Chain Emission Factors without Margins	Margins of Supply Chain Emission Factors	Supply Chain Emission Factors with Margins	Unnamed: 7	ReliabilityScore of Factors without Margins	DQ TemporalCorrelation of Factors without Margins	DQ Geograph
0	1111A0	Fresh soybeans, canola, flaxseeds, and other o...	carbon dioxide	kg/2018 USD, purchaser price	0.373	0.072	0.444	NaN	4	3	
1	1111A0	Fresh soybeans, canola, flaxseeds, and other o...	methane	kg/2018 USD, purchaser price	0.001	0.001	0.002	NaN	4	3	
2	1111A0	Fresh soybeans, canola, flaxseeds, and other o...	nitrous oxide	kg/2018 USD, purchaser price	0.002	0.000	0.002	NaN	4	3	
3	1111A0	Fresh soybeans, canola, flaxseeds, and other o...	other GHGs	kg CO2e/2018 USD, purchaser price	0.002	0.000	0.002	NaN	3	3	
4	1111B0	Fresh wheat, corn, rice, and other grains	carbon dioxide	kg/2018 USD, purchaser price	0.722	0.079	0.801	NaN	4	3	
...	...	...	...	...	...	...	...	...	...	...	
3151	813B00	Civic, social, professional, and similar organ...	other GHGs	kg CO2e/2018 USD, purchaser price	0.008	0.000	0.008	NaN	4	3	
3152	814000	Private households	carbon dioxide	kg/2018 USD, purchaser price	0.000	0.000	0.000	NaN	4	3	
3153	814000	Private households	methane	kg/2018 USD, purchaser price	0.000	0.000	0.000	NaN	4	3	
3154	814000	Private households	nitrous oxide	kg/2018 USD, purchaser price	0.000	0.000	0.000	NaN	4	3	
3155	814000	Private households	other GHGs	kg CO2e/2018 USD, purchaser price	0.000	0.000	0.000	NaN	4	3	

3156 rows × 15 columns

```
len(all_data)
```



7

```
df = pd.concat(all_data, ignore_index=True)
df.head(10)
```



	Code	Name	Substance	Unit	Supply Chain Emission Factors without Margins	Margins of Supply Chain Emission Factors	Supply Chain Emission Factors with Margins	Unnamed: 7	ReliabilityScore of Factors without Margins	DQ TemporalCorrelation of Factors without Margins	DQ Geographica of Fac
0	1111A0	Fresh soybeans, canola, flaxseeds, and other o...	carbon dioxide	kg/2018 USD, purchaser price	0.398	0.073	0.470	NaN	4	3	
1	1111A0	Fresh soybeans, canola, flaxseeds, and other o...	methane	kg/2018 USD, purchaser price	0.001	0.001	0.002	NaN	4	3	
2	1111A0	Fresh soybeans, canola, flaxseeds, and other o...	nitrous oxide	kg/2018 USD, purchaser price	0.002	0.000	0.002	NaN	4	3	
3	1111A0	Fresh soybeans, canola, flaxseeds, and other o...	other GHGs	kg CO2e/2018 USD, purchaser price	0.002	0.000	0.002	NaN	3	3	
4	1111B0	Fresh wheat, corn, rice, and other grains	carbon dioxide	kg/2018 USD, purchaser price	0.659	0.081	0.740	NaN	4	3	
5	1111B0	Fresh wheat, corn, rice, and other grains	methane	kg/2018 USD, purchaser price	0.008	0.001	0.009	NaN	2	3	
6	1111B0	Fresh wheat, corn, rice, and other grains	nitrous oxide	kg/2018 USD, purchaser price	0.004	0.000	0.004	NaN	4	3	
7	1111B0	Fresh wheat, corn, rice, and other grains	other GHGs	kg CO2e/2018 USD, purchaser price	0.004	0.000	0.004	NaN	3	3	
8	111200	Fresh vegetables, melons, and potatoes	carbon dioxide	kg/2018 USD, purchaser price	0.183	0.132	0.315	NaN	3	3	
9	111200	Fresh vegetables, melons, and potatoes	methane	kg/2018 USD, purchaser price	0.001	0.001	0.002	NaN	4	3	

len(df)

22092

Step 3: Data Preprocessing

df.columns

Index(['Code', 'Name', 'Substance', 'Unit',
'Supply Chain Emission Factors without Margins',

```
'Margins of Supply Chain Emission Factors',
'Supply Chain Emission Factors with Margins', 'Unnamed: 7',
'DQ ReliabilityScore of Factors without Margins',
'DQ TemporalCorrelation of Factors without Margins',
'DQ GeographicalCorrelation of Factors without Margins',
'DQ TechnologicalCorrelation of Factors without Margins',
'DQ DataCollection of Factors without Margins', 'Source', 'Year'],
dtype='object')
```

```
df.isnull().sum()
```

```

0
-----
Code                0
Name                0
Substance           0
Unit                0
Supply Chain Emission Factors without Margins  0
Margins of Supply Chain Emission Factors      0
Supply Chain Emission Factors with Margins    0
Unnamed: 7                22092
DQ ReliabilityScore of Factors without Margins  0
DQ TemporalCorrelation of Factors without Margins  0
DQ GeographicalCorrelation of Factors without Margins  0
DQ TechnologicalCorrelation of Factors without Margins  0
DQ DataCollection of Factors without Margins      0
Source                0
Year                  0
```

```
dtype: int64
```

Week 2 - 23 June 2025 - S4F

```
df.drop(columns=['Unnamed: 7'],inplace=True)
```

```
df.columns
```

```
Index(['Code', 'Name', 'Substance', 'Unit',
'Supply Chain Emission Factors without Margins',
'Margins of Supply Chain Emission Factors',
'Supply Chain Emission Factors with Margins',
'DQ ReliabilityScore of Factors without Margins',
'DQ TemporalCorrelation of Factors without Margins',
'DQ GeographicalCorrelation of Factors without Margins',
'DQ TechnologicalCorrelation of Factors without Margins',
'DQ DataCollection of Factors without Margins', 'Source', 'Year'],
dtype='object')
```

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22092 entries, 0 to 22091
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Code                                22092 non-null  object
1   Name                                22092 non-null  object
2   Substance                           22092 non-null  object
3   Unit                                22092 non-null  object
4   Supply Chain Emission Factors without Margins  22092 non-null  float64
5   Margins of Supply Chain Emission Factors      22092 non-null  float64
6   Supply Chain Emission Factors with Margins    22092 non-null  float64
7   DQ ReliabilityScore of Factors without Margins  22092 non-null  int64
8   DQ TemporalCorrelation of Factors without Margins  22092 non-null  int64
9   DQ GeographicalCorrelation of Factors without Margins  22092 non-null  int64
10  DQ TechnologicalCorrelation of Factors without Margins  22092 non-null  int64
11  DQ DataCollection of Factors without Margins      22092 non-null  int64
```

```
12 Source                22092 non-null object
13 Year                  22092 non-null int64
dtypes: float64(3), int64(6), object(5)
memory usage: 2.4+ MB
None
```

```
df.describe().T
```



	count	mean	std	min	25%	50%	75%	max
Supply Chain Emission Factors without Margins	22092.0	0.084807	0.267039	0.0	0.0	0.002	0.044	7.228
Margins of Supply Chain Emission Factors	22092.0	0.012857	0.078720	0.0	0.0	0.000	0.000	3.349
Supply Chain Emission Factors with Margins	22092.0	0.097681	0.288992	0.0	0.0	0.003	0.052	7.290
DQ ReliabilityScore of Factors without Margins	22092.0	3.308030	0.499643	2.0	3.0	3.000	4.000	4.000
DQ TemporalCorrelation of Factors without Margins	22092.0	2.571429	0.494883	2.0	2.0	3.000	3.000	3.000
DQ GeographicalCorrelation of Factors without Margins	22092.0	1.000000	0.000000	1.0	1.0	1.000	1.000	1.000
DQ TechnologicalCorrelation of Factors without Margins	22092.0	2.632129	1.135661	1.0	1.0	3.000	3.000	5.000
DQ DataCollection of Factors without Margins	22092.0	1.000000	0.000000	1.0	1.0	1.000	1.000	1.000
Year	22092.0	2013.000000	2.000045	2010.0	2011.0	2013.000	2015.000	2016.000

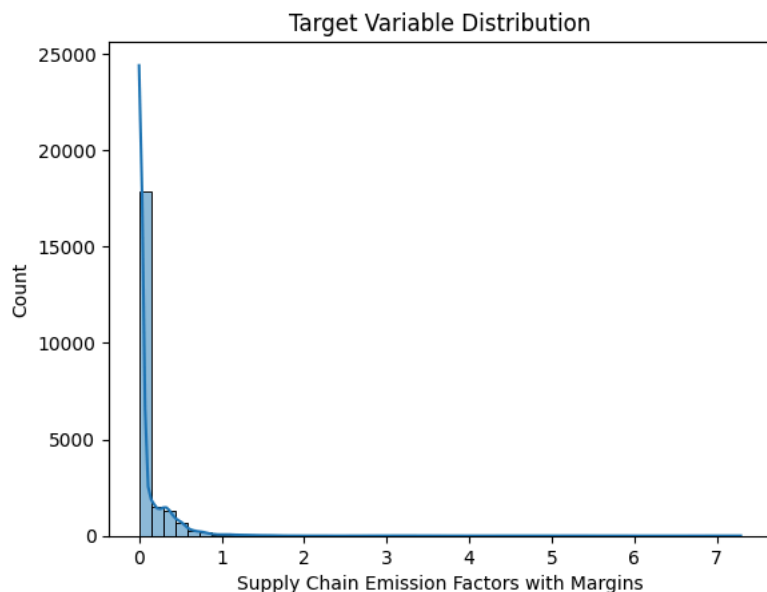
```
df.isnull().sum()
```



	0
Code	0
Name	0
Substance	0
Unit	0
Supply Chain Emission Factors without Margins	0
Margins of Supply Chain Emission Factors	0
Supply Chain Emission Factors with Margins	0
DQ ReliabilityScore of Factors without Margins	0
DQ TemporalCorrelation of Factors without Margins	0
DQ GeographicalCorrelation of Factors without Margins	0
DQ TechnologicalCorrelation of Factors without Margins	0
DQ DataCollection of Factors without Margins	0
Source	0
Year	0

dtype: int64

```
sns.histplot(df['Supply Chain Emission Factors with Margins'], bins=50, kde=True)
plt.title('Target Variable Distribution')
plt.show()
```



```
print(df['Substance'].value_counts())
```



```
Substance
carbon dioxide    5523
methane           5523
nitrous oxide     5523
other GHGs        5523
Name: count, dtype: int64
```

```
print(df['Unit'].value_counts())
```



```
Unit
kg/2018 USD, purchaser price    16569
kg CO2e/2018 USD, purchaser price    5523
Name: count, dtype: int64
```

```
print(df['Unit'].unique())
```



```
['kg/2018 USD, purchaser price' 'kg CO2e/2018 USD, purchaser price']
```

```
print(df['Source'].value_counts())
```



```
Source
Industry    11060
Commodity   11032
Name: count, dtype: int64
```

```
df['Substance'].unique()
```



```
array(['carbon dioxide', 'methane', 'nitrous oxide', 'other GHGs'],
      dtype=object)
```

```
substance_map={'carbon dioxide':0, 'methane':1, 'nitrous oxide':2, 'other GHGs':3}
```

```
df['Substance']=df['Substance'].map(substance_map)
```

```
df['Substance'].unique()
```



```
array([0, 1, 2, 3])
```



```
print(df['Unit'].unique())
```

```
↳ ['kg/2018 USD, purchaser price' 'kg CO2e/2018 USD, purchaser price']
```

```
unit_map={'kg/2018 USD, purchaser price':0, 'kg CO2e/2018 USD, purchaser price':1}
```

```
df['Unit']=df['Unit'].map(unit_map)
```

```
print(df['Unit'].unique())
```

```
↳ [0 1]
```

```
print(df['Source'].unique())
```

```
↳ ['Commodity' 'Industry']
```

```
source_map={'Commodity':0, 'Industry':1}
```

```
df['Source']=df['Source'].map(source_map)
```

```
print(df['Source'].unique())
```

```
↳ [0 1]
```

```
df.info()
```

```
↳ <class 'pandas.core.frame.DataFrame'>
RangeIndex: 22092 entries, 0 to 22091
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Code                                  22092 non-null  object
1   Name                                  22092 non-null  object
2   Substance                             22092 non-null  int64
3   Unit                                  22092 non-null  int64
4   Supply Chain Emission Factors without Margins  22092 non-null  float64
5   Margins of Supply Chain Emission Factors      22092 non-null  float64
6   Supply Chain Emission Factors with Margins    22092 non-null  float64
7   DQ ReliabilityScore of Factors without Margins  22092 non-null  int64
8   DQ TemporalCorrelation of Factors without Margins  22092 non-null  int64
9   DQ GeographicalCorrelation of Factors without Margins  22092 non-null  int64
10  DQ TechnologicalCorrelation of Factors without Margins  22092 non-null  int64
11  DQ DataCollection of Factors without Margins    22092 non-null  int64
12  Source                                           22092 non-null  int64
13  Year                                             22092 non-null  int64
dtypes: float64(3), int64(9), object(2)
memory usage: 2.4+ MB
```

```
df.Code.unique()
```

```
↳ array(['1111A0', '1111B0', '111200', '111300', '111400', '111900',
        '112120', '1121A0', '112300', '112A00', '113000', '114000',
        '115000', '211000', '212100', '212230', '2122A0', '212310',
        '2123A0', '213111', '21311A', '221100', '221200', '221300',
        '230301', '230302', '233210', '233230', '233240', '233262',
        '2332A0', '2332C0', '2332D0', '233411', '233412', '2334A0',
        '311111', '311119', '311210', '311221', '311224', '311225',
        '311230', '311300', '311410', '311420', '311513', '311514',
        '31151A', '311520', '311615', '31161A', '311700', '311810',
        '3118A0', '311910', '311920', '311930', '311940', '311990',
        '312110', '312120', '312130', '312140', '312200', '313100',
        '313200', '313300', '314110', '314120', '314900', '315000',
        '316000', '321100', '321200', '321910', '3219A0', '322110',
        '322120', '322130', '322210', '322220', '322230', '322291',
        '322299', '323110', '323120', '324110', '324121', '324122',
        '324190', '325110', '325120', '325130', '325180', '325190',
        '325211', '3252A0', '325310', '325320', '325411', '325412',
```

```
'325413', '325414', '325510', '325520', '325610', '325620',
'325910', '3259A0', '326110', '326120', '326130', '326140',
'326150', '326160', '326190', '326210', '326220', '326290',
'327100', '327200', '327310', '327320', '327330', '327390',
'327400', '327910', '327991', '327992', '327993', '327999',
'331110', '331200', '331313', '33131B', '331410', '331420',
'331490', '331510', '331520', '332114', '332119', '33211A',
'332200', '332310', '332320', '332410', '332420', '332430',
'332500', '332600', '332710', '332720', '332800', '332913',
'33291A', '332991', '332996', '332999', '33299A', '333111',
'333112', '333120', '333130', '333242', '33329A', '333314',
'333316', '333318', '333413', '333414', '333415', '333511',
'333514', '333517', '33351B', '333611', '333612', '333613',
'333618', '333912', '33391A', '333920', '333991', '333993',
'333994', '33399A', '33399B', '334111', '334112', '334118',
'334210', '334220', '334290', '334300', '334413', '334418',
'33441A', '334510', '334511', '334512', '334513', '334514',
'334515', '334516', '334517', '33451A', '334610', '335110',
'335120', '335210', '335221', '335222', '335224', '335228',
'335311', '335312', '335313', '335314', '335911', '335912',
'335920', '335930', '335991', '335999', '336111', '336112',
'336120', '336211', '336212', '336213', '336214', '336310',
'336320', '336350', '336360', '336370', '336390', '3363A0',
'336411', '336412', '336413', '336414', '33641A', '336500',
'336611', '336612', '336991', '336992', '336999', '337110',
'337121', '337122', '337127', '33712N', '337215', '33721A',
'337900', '339112', '339113', '339114', '339115', '339116',
'339910', '339920', '339930', '339940', '339950', '339990',
'4200ID', '423100', '423400', '423600', '423800', '423A00',
'424200', '424400', '424700', '424A00', '425000', '441000',
'444000', '445000', '446000', '447000', '448000', '452000',
'454000', '481000', '482000', '483000', '484000', '485000',
'486000', '48A000', '491000', '492000', '493000', '480000',
'511110', '511120', '511130', '5111A0', '511200', '512100',
'512200', '515100', '515200', '517110', '517210', '517A00',
'518200', '519130', '5191A0', '522A00', '523900', '523A00',
'524113', '5241XX', '524200', '525000', '52A000', '531HSO',
'531HST', '531ORE', '532100', '532400', '532A00', '533000',
'541100', '541200', '541300', '541400', '541511', '541512',
'54151A', '541610', '5416A0', '541700', '541800', '541920',
'541940', '5419A0', '550000', '561100', '561200', '561300',
```

```
df.Name.unique()
```

```
array(['Fresh soybeans, canola, flaxseeds, and other oilseeds',
      'Fresh wheat, corn, rice, and other grains',
      'Fresh vegetables, melons, and potatoes',
      'Fresh fruits and tree nuts',
      'Greenhouse crops, mushrooms, nurseries, and flowers',
      'Tobacco, cotton, sugarcane, peanuts, sugar beets, herbs and spices, and other crops',
      'Dairies', 'Cattle ranches and feedlots', 'Poultry farms',
      'Animal farms and aquaculture ponds (except cattle and poultry)',
      'Timber and raw forest products', 'Wild-caught fish and game',
      'Agriculture and forestry support', 'Unrefined oil and gas',
      'Coal', 'Copper, nickel, lead, and zinc',
      'Iron, gold, silver, and other metal ores', 'Dimensional stone',
      'Sand, gravel, clay, phosphate, other nonmetallic minerals',
      'Well drilling', 'Other support activities for mining',
      'Electricity', 'Natural gas',
      'Drinking water and wastewater treatment',
      'Nonresidential maintenance and repair',
      'Residential maintenance and repair', 'Health care structures',
      'Manufacturing structures', 'Power and communication structures',
      'Educational and vocational structures',
      'Commercial structures, including farm structures',
      'Transportation structures and highways and streets',
      'Other nonresidential structures',
      'Single-family residential structures',
      'Multifamily residential structures',
      'Other residential structures', 'Dog and cat food',
      'Other animal food', 'Flours and malts', 'Corn products',
      'Soybean and other oilseed processing',
      'Refined vegetable, olive, and seed oils', 'Breakfast cereals',
      'Sugar, candy, and chocolate', 'Frozen food',
      'Fruit and vegetable preservation', 'Cheese',
      'Dry, condensed, and evaporated dairy', 'Fluid milk and butter',
      'Ice cream and frozen desserts', 'Packaged poultry',
      'Packaged meat (except poultry)', 'Seafood',
      'Bread and other baked goods',
      'Cookies, crackers, pastas, and tortillas', 'Snack foods',
      'Coffee and tea', 'Flavored drink concentrates',
      'Seasonings and dressings', 'All other foods',
      'Soft drinks, bottled water, and ice', 'Breweries and beer',
```

```
'Wineries and wine', 'Distilleries and spirits',
'Tobacco products', 'Fiber, yarn, and thread', 'Fabric',
'Finished and coated fabric', 'Carpets and rugs',
'Curtains and linens', 'Other textiles', 'Clothing', 'Leather',
'Lumber and treated lumber', 'Plywood and veneer',
'Wooden windows, door, and flooring',
'Veneer, plywood, and engineered wood', 'Wood pulp', 'Paper',
'Cardboard', 'Cardboard containers', 'Paper bags and coated paper',
'Stationery', 'Sanitary paper (tissues, napkins, diapers, etc.)',
'All other converted paper products',
'Books, newspapers, magazines, and other print media',
'Printing support',
'Gasoline, fuels, and by-products of petroleum refining',
'Asphalt pavement', 'Asphalt shingles',
'Other petroleum and coal products', 'Petrochemicals',
'Compressed Gases', 'Synthetic dyes and pigments',
'Other basic inorganic chemicals', 'Other basic organic chemicals',
'Plastics', 'Synthetic rubber and artificial and synthetic fibers',
'Fertilizers', 'Pesticides', 'Medicinal and botanical ingredients',
```

```
len(df.Name.unique())
```

 713

Top 10 Emmiting Industry

```
top_emitters = df[['Name', 'Supply Chain Emission Factors with Margins']].groupby('Name').mean().sort_values(
    'Supply Chain Emission Factors with Margins', ascending=False).head(10)
```

```
top_emitters = top_emitters.reset_index()
```

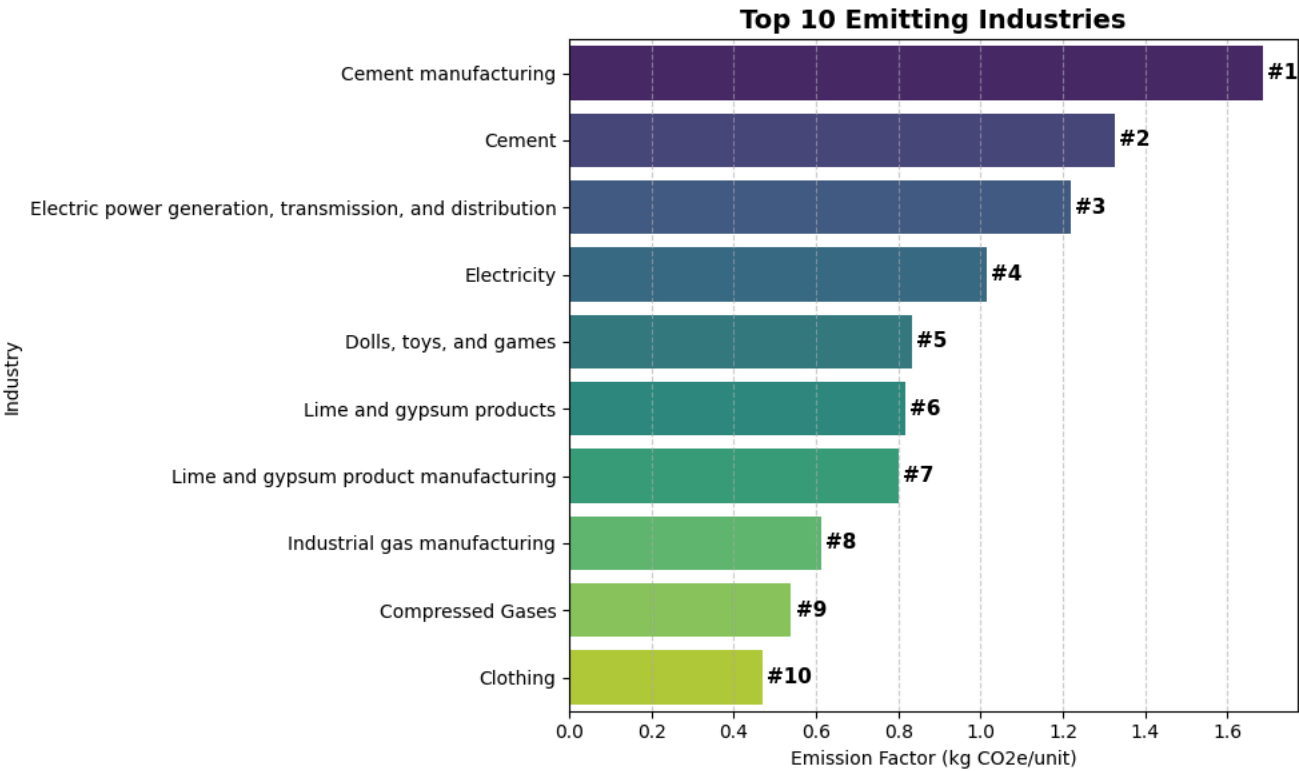
top_emitters



	Name	Supply Chain Emission Factors with Margins
0	Cement manufacturing	1.686179
1	Cement	1.324964
2	Electric power generation, transmission, and d...	1.220357
3	Electricity	1.016143
4	Dolls, toys, and games	0.832179
5	Lime and gypsum products	0.816536
6	Lime and gypsum product manufacturing	0.799679
7	Industrial gas manufacturing	0.612929
8	Compressed Gases	0.539679
9	Clothing	0.468714

```
plt.figure(figsize=(10,6))
sns.barplot(
    x='Supply Chain Emission Factors with Margins',
    y='Name',
    data=top_emitters,
    hue='Name',
    palette='viridis'
)
for i, (value, name) in enumerate(zip(top_emitters['Supply Chain Emission Factors with Margins'], top_emitters.index), start=1):
    plt.text(value + 0.01, i - 1, f'#{i}', va='center', fontsize=11, fontweight='bold', color='black')

plt.title('Top 10 Emitting Industries', fontsize=14, fontweight='bold')
plt.xlabel('Emission Factor (kg CO2e/unit)')
plt.ylabel('Industry')
plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.tight_layout()
```



```
df.drop(columns=['Name', 'Code', 'Year'], inplace=True)
```

```
df.head(1)
```



Substance	Unit	Supply Chain Emission Factors without Margins	Margins of Supply Chain Emission Factors	Supply Chain Emission Factors with Margins	DQ ReliabilityScore of Factors without Margins	DQ TemporalCorrelation of Factors without Margins	DQ GeographicalCorrelation of Factors without Margins	DQ TechnologicalCorrelation of Factors without Margins	DQ DataCollection of Factors without Margins	Source
0	0	0	0.398	0.073	0.47	4	3	1		

```
df.shape
```



(22092, 11)

```
df.columns
```



```
Index(['Substance', 'Unit', 'Supply Chain Emission Factors without Margins',  
      'Margins of Supply Chain Emission Factors',  
      'Supply Chain Emission Factors with Margins',  
      'DQ ReliabilityScore of Factors without Margins',  
      'DQ TemporalCorrelation of Factors without Margins',  
      'DQ GeographicalCorrelation of Factors without Margins',  
      'DQ TechnologicalCorrelation of Factors without Margins',  
      'DQ DataCollection of Factors without Margins', 'Source'],  
      dtype='object')
```

```
X = df.drop(columns=['Supply Chain Emission Factors with Margins'])  
y = df['Supply Chain Emission Factors with Margins']
```

```
X.head()
```

	Substance	Unit	Supply Chain Emission Factors without Margins	Margins of Supply Chain Emission Factors	ReliabilityScore of Factors without Margins	DQ TemporalCorrelation of Factors without Margins	DQ GeographicalCorrelation of Factors without Margins	DQ TechnologicalCorrelation of Factors without Margins	DQ DataCo: of
0	0	0	0.398	0.073	4	3	1	4	
1	1	0	0.001	0.001	4	3	1	1	
2	2	0	0.002	0.000	4	3	1	4	
3	3	1	0.002	0.000	3	3	1	3	
4	0	0	0.659	0.081	4	3	1	4	

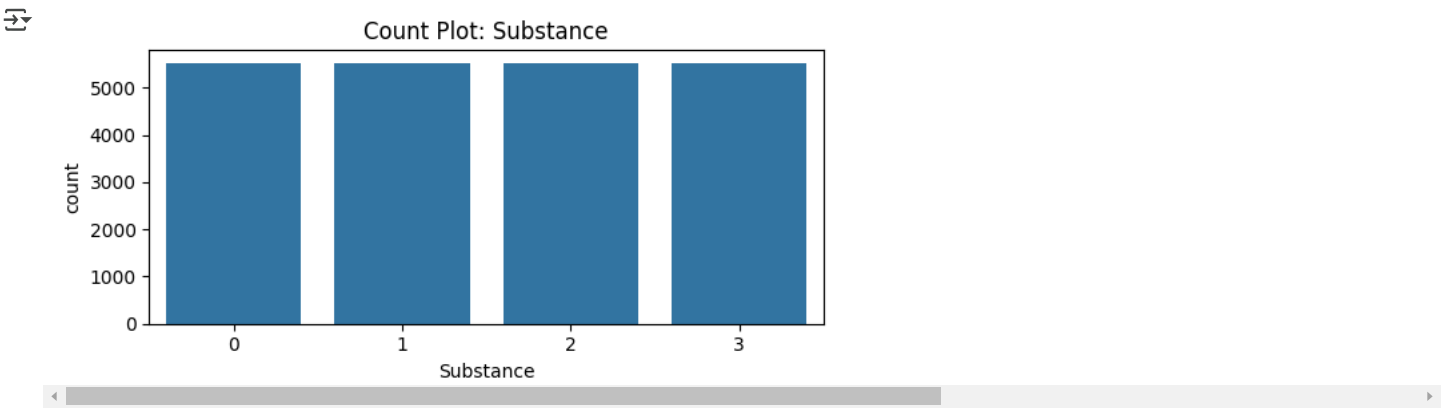
```
y.head()
```

Supply Chain Emission Factors with Margins	
0	0.470
1	0.002
2	0.002
3	0.002
4	0.740

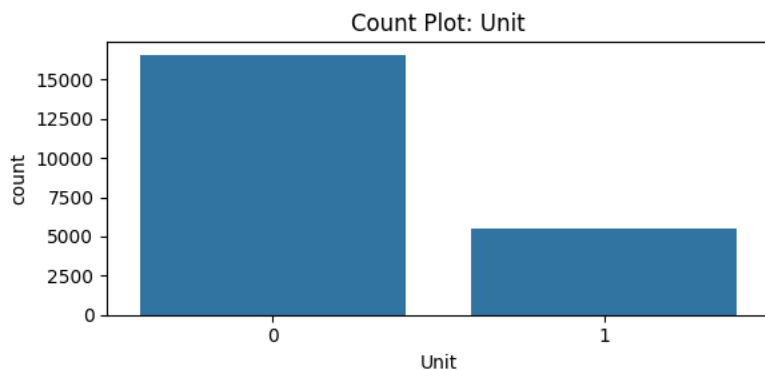
dtype: float64

Univariate Analysis

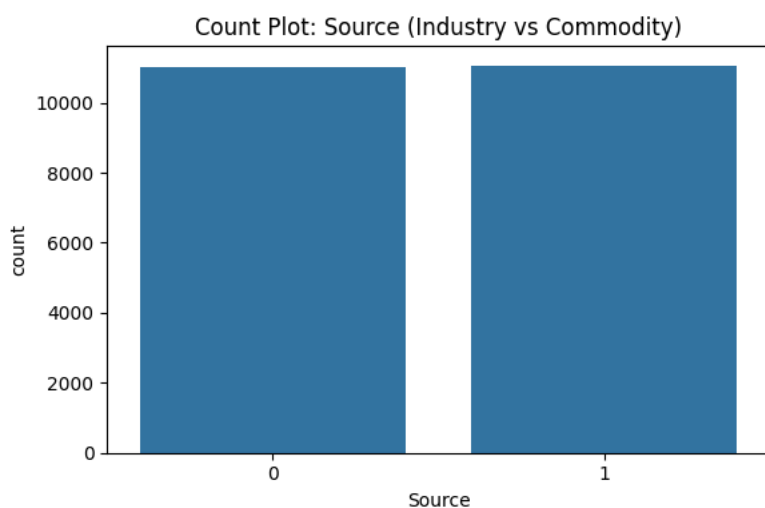
```
plt.figure(figsize=(6, 3))
sns.countplot(x=df["Substance"])
plt.title("Count Plot: Substance")
plt.xticks()
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(6, 3))
sns.countplot(x=df["Unit"])
plt.title("Count Plot: Unit")
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(6, 4))
sns.countplot(x=df["Source"])
plt.title("Count Plot: Source (Industry vs Commodity)")
plt.tight_layout()
plt.show()
```



```
df.columns
```



```
Index(['Substance', 'Unit', 'Supply Chain Emission Factors without Margins',
      'Margins of Supply Chain Emission Factors',
      'Supply Chain Emission Factors with Margins',
      'DQ ReliabilityScore of Factors without Margins',
      'DQ TemporalCorrelation of Factors without Margins',
      'DQ GeographicalCorrelation of Factors without Margins',
      'DQ TechnologicalCorrelation of Factors without Margins',
      'DQ DataCollection of Factors without Margins', 'Source'],
      dtype='object')
```

Multivariate Anslsysis

```
df.select_dtypes(include=np.number).corr()
```



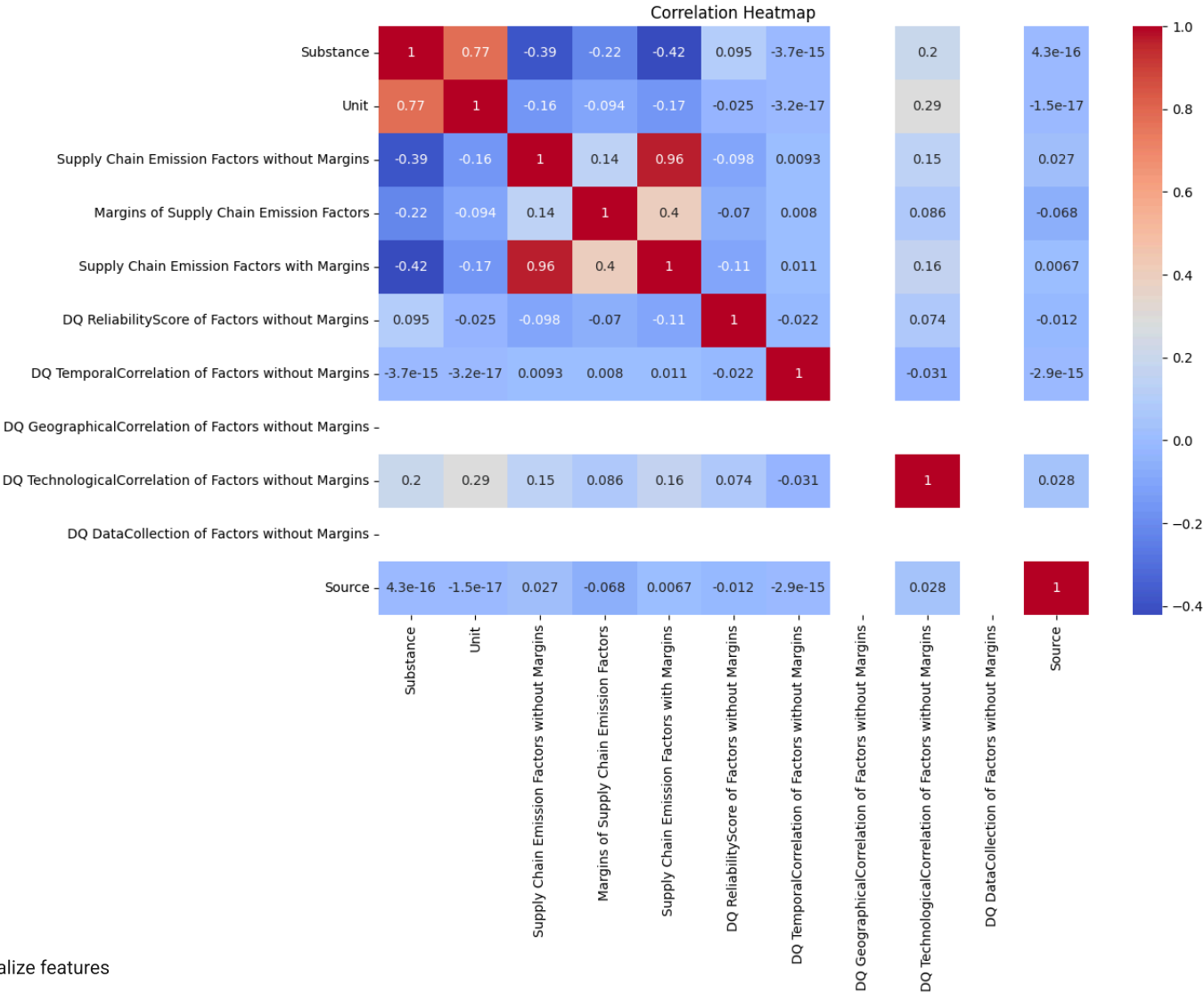
	Substance	Unit	Supply Chain Emission Factors without Margins	Margins of Supply Chain Emission Factors	Supply Chain Emission Factors with Margins	DQ ReliabilityScore of Factors without Margins	DQ TemporalCorrelation of Factors without Margins	DQ GeographicalCorrelation of Factors without Margins
Substance	1.000000e+00	7.745967e-01	-0.391851	-0.218400	-0.421603	0.095092	-3.667637e-15	
Unit	7.745967e-01	1.000000e+00	-0.155859	-0.094300	-0.169741	-0.025159	-3.173071e-17	
Supply Chain Emission Factors without Margins	-3.918505e-01	-1.558594e-01	1.000000	0.143005	0.962971	-0.098000	9.283870e-03	
Margins of Supply Chain Emission Factors	-2.184002e-01	-9.429989e-02	0.143005	1.000000	0.404541	-0.069598	7.953109e-03	
Supply Chain Emission Factors with Margins	-4.216032e-01	-1.697410e-01	0.962971	0.404541	1.000000	-0.109494	1.074848e-02	
DQ ReliabilityScore of Factors without Margins	9.509190e-02	-2.515938e-02	-0.098000	-0.069598	-0.109494	1.000000	-2.170715e-02	
DQ TemporalCorrelation of Factors without Margins	-3.667637e-15	-3.173071e-17	0.009284	0.007953	0.010748	-0.021707	1.000000e+00	
DQ GeographicalCorrelation of Factors without Margins	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
DQ TechnologicalCorrelation of Factors without Margins	1.984154e-01	2.869901e-01	0.148410	0.086335	0.160574	0.073583	-3.060675e-02	
DQ DataCollection of Factors without Margins	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
Source	4.273306e-16	-1.545892e-17	0.027131	-0.067504	0.006688	-0.012287	-2.909814e-15	

```
df.info()
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22092 entries, 0 to 22091
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Substance                             22092 non-null  int64
1   Unit                                  22092 non-null  int64
2   Supply Chain Emission Factors without Margins  22092 non-null  float64
3   Margins of Supply Chain Emission Factors      22092 non-null  float64
4   Supply Chain Emission Factors with Margins    22092 non-null  float64
5   DQ ReliabilityScore of Factors without Margins  22092 non-null  int64
6   DQ TemporalCorrelation of Factors without Margins  22092 non-null  int64
7   DQ GeographicalCorrelation of Factors without Margins  22092 non-null  int64
8   DQ TechnologicalCorrelation of Factors without Margins  22092 non-null  int64
9   DQ DataCollection of Factors without Margins    22092 non-null  int64
10  Source                                  22092 non-null  int64
dtypes: float64(3), int64(8)
memory usage: 1.9 MB

plt.figure(figsize=(12, 8))
sns.heatmap(df.select_dtypes(include=np.number).corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



Normalize features

```
X.describe().T
```



	count	mean	std	min	25%	50%	75%	max
Substance	22092.0	1.500000	1.118059	0.0	0.75	1.500	2.250	3.000
Unit	22092.0	0.250000	0.433023	0.0	0.00	0.000	0.250	1.000
Supply Chain Emission Factors without Margins	22092.0	0.084807	0.267039	0.0	0.00	0.002	0.044	7.228
Margins of Supply Chain Emission Factors	22092.0	0.012857	0.078720	0.0	0.00	0.000	0.000	3.349
DQ ReliabilityScore of Factors without Margins	22092.0	3.308030	0.499643	2.0	3.00	3.000	4.000	4.000
DQ TemporalCorrelation of Factors without Margins	22092.0	2.571429	0.494883	2.0	2.00	3.000	3.000	3.000
DQ GeographicalCorrelation of Factors without Margins	22092.0	1.000000	0.000000	1.0	1.00	1.000	1.000	1.000
DQ TechnologicalCorrelation of Factors without Margins	22092.0	2.632129	1.135661	1.0	1.00	3.000	3.000	5.000
DQ DataCollection of Factors without Margins	22092.0	1.000000	0.000000	1.0	1.00	1.000	1.000	1.000
Source	22092.0	0.500634	0.500011	0.0	0.00	1.000	1.000	1.000

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
X_scaled[0].min(),X_scaled[0].max()
```



```
(np.float64(-1.3416407864998738), np.float64(1.3849614361966767))
```



```
np.round(X_scaled.mean()), np.round(X_scaled.std())
```

```
↵ (np.float64(-0.0), np.float64(1.0))
```

Divide the data into train and test

```
X.shape
```

```
↵ (22092, 10)
```